

OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 09

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

QUESTION: 1

Define an abstract class ArrayMultiplier with a pure virtual function calculate(). Then, create two derived classes: ArrayMultiplier1D for 1D array multiplication and ArrayMultiplier2D for 2D array multiplication. In the calculate() function of each derived class, perform the multiplication operation based on the specific array dimension.

SOURCECODE:

```
#include <iostream>
using namespace std;

// Abstract Class
class ArrayMultiplier {
public:
    virtual void calculate() = 0;
};

// 1D Array
class ArrayMultiplier1D : public ArrayMultiplier {
    int arr[3];

public:
    ArrayMultiplier1D(int a, int b, int c) {
        arr[0] = a; arr[1] = b; arr[2] = c;
    }

    void calculate() {
        int result = 1;
        for (int i = 0; i < 3; i++)
            result *= arr[i];
    }
}
```

```

        cout << "1D Multiplication: " << result << endl;
    }
};

// 2D Array
class ArrayMultiplier2D : public ArrayMultiplier {
    int arr[2][2];

public:
    ArrayMultiplier2D(int a, int b, int c, int d) {
        arr[0][0] = a; arr[0][1] = b;
        arr[1][0] = c; arr[1][1] = d;
    }

    void calculate() {
        int result = 1;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                result *= arr[i][j];

        cout << "2D Multiplication: " << result << endl;
    }
};

int main() {
    ArrayMultiplier *a1 = new ArrayMultiplier1D(2,3,4);
    ArrayMultiplier *a2 = new ArrayMultiplier2D(1,2,3,4);

    a1->calculate();
    a2->calculate();

    return 0;
}

```

OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 9(1).exe
1D Multiplication: 24
2D Multiplication: 24

-----
Process exited after 0.4291 seconds with return value 0
Press any key to continue . . .
```

QUESTION: 2

Write a program to calculate final bill after discount. “ImtiazStore” gives 7 percent discount on total_bill while “BinHashimStore” gives 5 percent discount on total_bill. You have to initialize value of total_bill through a constructor and then calculate final bill after discount for both stores using the concept of abstract class and virtual functions.

SOURCECODE:

```
#include <iostream>
using namespace std;

class Store {
protected:
    double total_bill;

public:
    Store(double b) {
        total_bill = b;
    }

    virtual void calculate() = 0;
};

class ImtiazStore : public Store {
```

```

public:
    ImtiazStore(double b) : Store(b) {}

    void calculate() {
        double final = total_bill - (0.07 * total_bill);
        cout << "Imtiaz Final Bill: " << final << endl;
    }
};

class BinHashimStore : public Store {
public:
    BinHashimStore(double b) : Store(b) {}

    void calculate() {
        double final = total_bill - (0.05 * total_bill);
        cout << "BinHashim Final Bill: " << final << endl;
    }
};

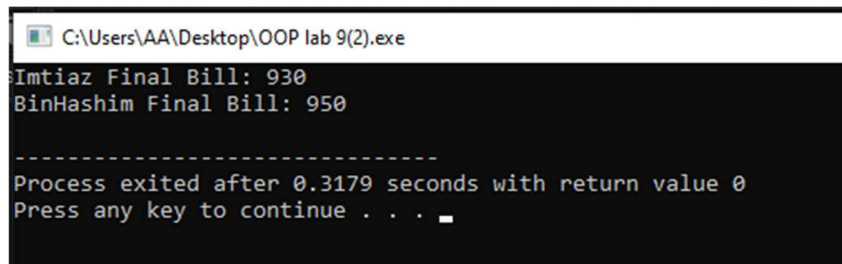
int main() {
    Store *s1 = new ImtiazStore(1000);
    Store *s2 = new BinHashimStore(1000);

    s1->calculate();
    s2->calculate();

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\OOP lab 9(2).exe
Imtiaz Final Bill: 930
BinHashim Final Bill: 950

-----
Process exited after 0.3179 seconds with return value 0
Press any key to continue . . . _

```

QUESTION: 3

You are tasked with developing a car rental system for a company. The system should allow customers to rent cars based on availability and manage the inventory of cars. To implement this system, you decide to use object-oriented programming and apply the concept of abstraction.

- **Abstract Base Class: Vehicle** - You create an abstract base class called Vehicle, which represents a generic vehicle in the rental system. The Vehicle class contains common attributes like carId, brand, and model. It also defines pure virtual functions such as isAvailable() and rent(), which represent the availability of the vehicle and the process of renting it. Since these functions are pure virtual, they have no implementation in the base class.
- **Derived Class: Car** - You derive a Car class from the Vehicle class, representing a car in the rental system. The Car class provides concrete implementations for the pure virtual functions isAvailable() and rent(). In the isAvailable() function, you check the availability of the car by querying its specific availability attribute. In the rent() function, you update the availability status of the car after it has been rented.
- **Rental System Class: RentalSystem** - You create a RentalSystem class responsible for managing the rental operations. This class interacts with the Vehicle objects through the base class interface. It has functions like rentVehicle() and returnVehicle(), which take a Vehicle object as a parameter and call the corresponding isAvailable() and rent() functions. The RentalSystem class can handle rental operations for any type of Vehicle, as long as it inherits from the Vehicle base class.
- **Customer Class: Customer** - You create a Customer class to represent the customers of the rental system. The Customer class has methods like rentVehicle() and returnVehicle(), which take a Vehicle object as a parameter. These methods interact with the RentalSystem class to perform the rental operations without knowing the specific type of the vehicle. This allows for flexibility and extensibility if new types of vehicles are added to the system in the future.

By utilizing pure virtual functions, you create an abstraction that defines the common interface and behavior for all vehicles in the system. The derived classes provide concrete implementations specific to their type of vehicle. The

RentalSystem class and the Customer class can interact with vehicles through the abstract base class interface, allowing for polymorphism and decoupling from the specific implementations. This abstraction simplifies the codebase, promotes modularity, and allows for easy addition of new vehicle types without modifying existing classes. Write a test program by defining an array of pointers of the Base class and dynamic memory for allocating memory to objects created.

SOURCECODE:

```
#include <iostream>
using namespace std;

// Abstract Base
class Vehicle {
protected:
    int carId;
    string brand;
    string model;

public:
    Vehicle(int id, string b, string m) {
        carId = id;
        brand = b;
        model = m;
    }

    virtual bool isAvailable() = 0;
    virtual void rent() = 0;
};

// Car Class
class Car : public Vehicle {
    bool available;

public:
```

```

Car(int id, string b, string m, bool a)
    : Vehicle(id, b, m) {
    available = a;
}

bool isAvailable() {
    return available;
}

void rent() {
    if (available) {
        available = false;
        cout << "Car rented\n";
    } else {
        cout << "Not available\n";
    }
}
};

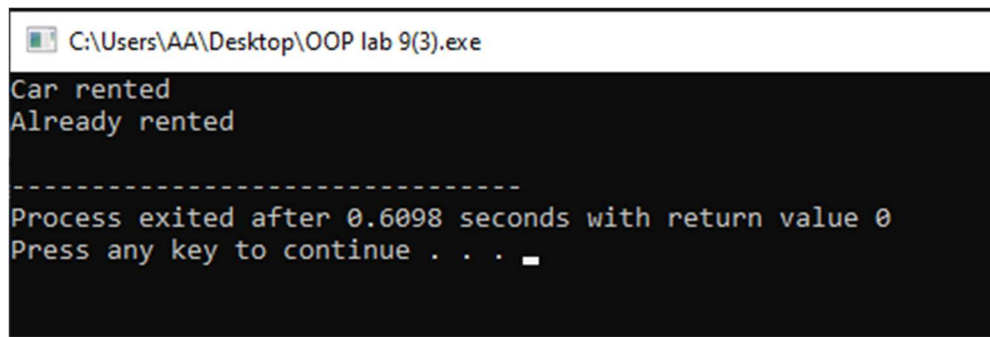
// Rental System
class RentalSystem {
public:
    void rentVehicle(Vehicle *v) {
        if (v->isAvailable())
            v->rent();
        else
            cout << "Already rented\n";
    }
};

// Customer
class Customer {
public:
    void rentVehicle(RentalSystem &rs, Vehicle *v) {
        rs.rentVehicle(v);
    }
};

```

```
    }  
};  
  
int main() {  
    Vehicle *v[2];  
  
    v[0] = new Car(1, "Toyota", "Corolla", true);  
    v[1] = new Car(2, "Honda", "Civic", true);  
  
    RentalSystem rs;  
    Customer c;  
  
    c.rentVehicle(rs, v[0]);  
    c.rentVehicle(rs, v[0]); // test again  
  
    return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\OOP lab 9(3).exe  
Car rented  
Already rented  
-----  
Process exited after 0.6098 seconds with return value 0  
Press any key to continue . . .
```

QUESTION: 4

Suppose you have to send an encrypted message over the network. There are two techniques that you can use to encrypt your message. Suppose your message is "Hello". It will be encrypted as follows according to those 2 techniques.

Technique 1: This technique simply converts alphabets into their respective ASCII codes.	H=72 E=69 L=76 L=76 O=79	Now the string will become, 7269767679
Technique 2: This technique simply converts alphabets into their respective ASCII code and then adding 2.	H=72+2 E=69+2 L=76+2 L=76+2 O=79+2	Now the string will become, 7471787881

Make use of an abstract class EncryptionTechnique and pure virtual function encrypt and implement this scenario. Make derived classes EncryptionTechnique1 and EncryptionTechnique2. Make a test program that takes an input string and encrypts it using both encryption techniques calling their respective encrypt functions.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class EncryptionTechnique {
public:
    virtual void encrypt(string msg) = 0;
};
```

```
// Technique 1
```

```
class Encryption1 : public EncryptionTechnique {
public:
    void encrypt(string msg) {
        cout << "Technique 1: ";
        for (int i = 0; i < msg.length(); i++)
            cout << int(msg[i]);
```

```

        cout << endl;
    }
};

// Technique 2
class Encryption2 : public EncryptionTechnique {
public:
    void encrypt(string msg) {
        cout << "Technique 2: ";
        for (int i = 0; i < msg.length(); i++)
            cout << int(msg[i]) + 2;
        cout << endl;
    }
};

int main() {
    string msg;
    cout << "Enter message: ";
    cin >> msg;

    EncryptionTechnique *e1 = new Encryption1();
    EncryptionTechnique *e2 = new Encryption2();

    e1->encrypt(msg);
    e2->encrypt(msg);

    return 0;
}

```

OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 9(4).exe
Enter message: hii, i'm a programmer
Technique 1: 10410510544
Technique 2: 10610710746

-----
Process exited after 52.43 seconds with return value 0
Press any key to continue . . .
```

QUESTION: 5

Design the corresponding decryption techniques and repeat Q3

SOURCECODE:

```
#include <iostream>
using namespace std;

class DecryptionTechnique {
public:
    virtual void decrypt(string msg) = 0;
};

// Technique 1
class Decrypt1 : public DecryptionTechnique {
public:
    void decrypt(string msg) {
        cout << "Decrypt 1: ";
        for (int i = 0; i < msg.length(); i += 2) {
            int num = stoi(msg.substr(i,2));
            cout << char(num);
        }
    }
}
```

```

        cout << endl;
    }
};

// Technique 2
class Decrypt2 : public DecryptionTechnique {
public:
    void decrypt(string msg) {
        cout << "Decrypt 2: ";
        for (int i = 0; i < msg.length(); i += 2) {
            int num = stoi(msg.substr(i,2));
            cout << char(num - 2);
        }
        cout << endl;
    }
};

int main() {
    string msg;
    cout << "Enter encrypted message: ";
    cin >> msg;

    DecryptionTechnique *d1 = new Decrypt1();
    DecryptionTechnique *d2 = new Decrypt2();

    d1->decrypt(msg);
    d2->decrypt(msg);

    return 0;
}

```

OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 9(5).exe
Enter encrypted message: i'm a programmer
Decrypt 1: terminate called after throwing an instance of 'std::invalid_argument'
  what():  stoi

-----
Process exited after 25 seconds with return value 3
Press any key to continue . . . _
```

SOLVED EXAMPLE

EXAMPLE: 1 (Abstract Class Example (Error Case))

SOURCECODE:

```
#include <iostream>
using namespace std;

class Test {
public:
    virtual void show() = 0; // pure virtual function
};

int main() {
    Test t; // ERROR: cannot create object of abstract class
    return 0;
}
```

OUTPUT:

```
// ERROR: cannot create object of abstract class
```

EXAMPLE: 2 (Abstract Class with Constructor (Working Example))

SOURCECODE:

```
#include <iostream>
using namespace std;

class Base {
protected:
    int x;
    virtual void fun() = 0;

public:
    Base(int i) {
        x = i;
    }
};

class Derived : public Base {
    int y;

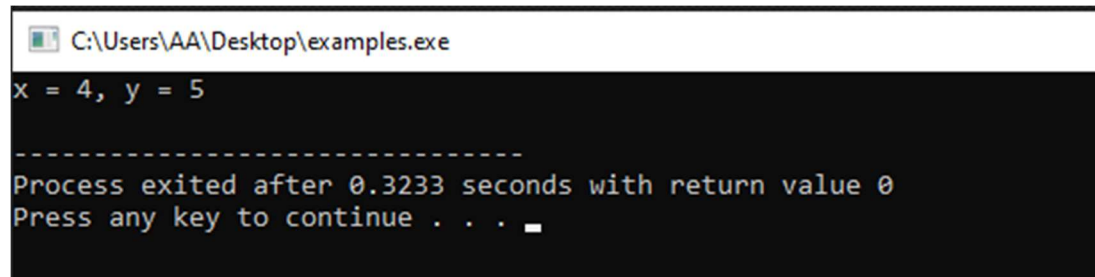
public:
    Derived(int i, int j) : Base(i) {
        y = j;
    }

    void fun() {
        cout << "x = " << x << ", y = " << y << endl;
    }
}
```

```
};

int main() {
    Derived d(4, 5);
    d.fun();
    return 0;
}
```

OUTPUT:



```
C:\Users\AA\Desktop\examples.exe
x = 4, y = 5
-----
Process exited after 0.3233 seconds with return value 0
Press any key to continue . . .
```

EXAMPLE: 3 (Abstract Class (Area Example – Square & Circle)

SOURCECODE:

```
#include <iostream>
using namespace std;

class Shape {
protected:
    float dimension;

public:
    void getDimension() {
        cin >> dimension;
    }

    virtual float calculateArea() = 0;
};
```

```
class Square : public Shape {
public:
    float calculateArea() {
        return dimension * dimension;
    }
};
```

```
class Circle : public Shape {
public:
    float calculateArea() {
        return 3.14 * dimension * dimension;
    }
};
```

```
int main() {
    Square s;
    Circle c;

    cout << "Enter side of square: ";
    s.getDimension();
    cout << "Area of square: " << s.calculateArea() << endl;

    cout << "\nEnter radius of circle: ";
    c.getDimension();
    cout << "Area of circle: " << c.calculateArea() << endl;

    return 0;
}
```

OUTPUT:

```
C:\Users\AA\Desktop\examples.exe
Enter side of square: 45.7
Area of square: 2088.49

Enter radius of circle: 23,6
Area of circle: 1661.06

-----
Process exited after 8.793 seconds with return value 0
Press any key to continue . . .
```